

ELA — Executive Life Assistant

Cosa stai costruendo

ELA è un assistente personale AI distribuito (Core Python, nodi Mac/Windows/iPhone).

La specifica completa è in ``docs/spec/ELA_spec.md``. È la fonte di verità: in caso di dubbio leggila. Non inventare comportamenti non descritti lì.

Regola fondamentale

Lavori su UNA milestone alla volta, quella indicata nel prompt. Non anticipare milestone future, non creare stub "per dopo", non aggiungere feature non richieste.

Ciclo obbligatorio: SPEC → IMPLEMENTATION → TEST → REVIEW → COMMIT → NEXT MILESTONE.

Tu ti fermi a REVIEW: il commit lo fa l'utente dopo revisione esterna.

Prima di scrivere codice

1. Leggi ``docs/spec/ELA_spec.md`` (sezioni rilevanti) e ``docs/adr/``.
2. Scrivi in ``docs/milestones/<id>.md`` la SPEC della milestone: cosa entra, cosa NON entra, criteri di accettazione, test previsti. Fermati e mostramela prima di implementare (usa plan mode).
3. Se la spec del documento è ambigua, elenca le interpretazioni possibili e chiedi. Non scegliere in silenzio.

Architettura (non negoziabile)

- ``src/ela/domain.py``: entità pure (pydantic), zero I/O, zero import da infrastruttura.
- ``src/ela/ports.py``: interfacce (Protocol/ABC). Nessun import di provider concreti.
- Il Core non importa mai ``anthropic``, ``openai``, ``sqlalchemy``, ``fastapi``, ``httpx``. Solo ``providers/``, ``infrastructure/``, ``api/`` possono farlo.
- Un Tool non può essere eseguito senza una ``PermissionDecision`` del Guardian. Il Guardian non possiede tool: decide su Capability + contesto.
- Capability (spec, rischio, schema, scope) ≠ Tool (implementazione).
- Task state machine deterministica: transizioni illegali sollevano eccezione.
- Audit Log append-only: nessuna API di update/delete.
- Fail-safe: in caso di dubbio l'azione è DENIED, mai eseguita.
- Nessun secret nel repo. Config via ``pydantic-settings`` + ``env``.

Qualità

- ``ruff check``, ``ruff format``, ``mypy --strict src/`` devono passare.
- ``pytest`` deve passare. Moduli security-critical (``permissions/``, ``audit/``, ``tasks/``): 100% branch coverage.
- Ogni nuova regola architetturale → un architecture test in ``tests/architecture/``.
- Ogni bug trovato → prima un test che fallisce, poi il fix.

Fine sessione

Produci un riepilogo con: file toccati, test aggiunti, output di ``pytest -q --cov``, cosa hai semplificato o lasciato aperto, domande per la review.